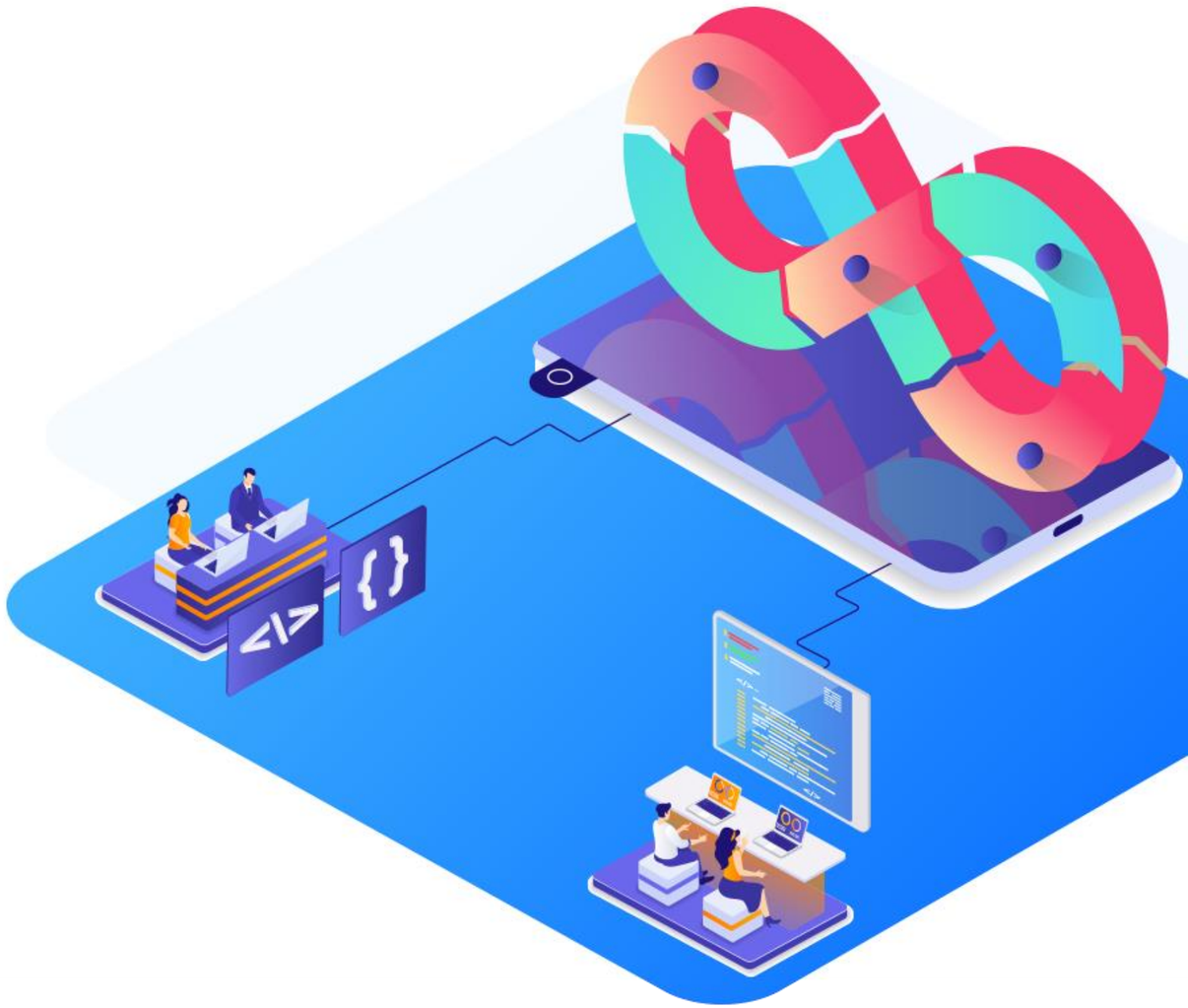
Configuration Management with Ansible and Terraform



Working with YAML

Learning Objectives

By the end of this lesson, you will be able to:

- Outline the fundamental concepts and components of YAML for understanding its application in configuration management
- Compare YAML with other data formats like JSON and XML to understand its unique features and benefits
- Create a YAML file to configure and validate the Apache web server setup for automated configuration management

Introduction to YAML

What Is Yet Another Markup Language (YAML)?

It is an easy-to-read data serialization language often used to create configuration files with various programming language.

YAML uses indentation to define the file structure, which makes the code easier to write and understand.

Purpose of Using YAML

Following are the key purposes of using YAML:

Human-readable
YAML files are easy for humans to read and write due to its simple syntax.

Whitespace indentation
It uses indentation for structure, like Python, making it visually clear.

Expressiveness
It supports lists and dictionaries, suitable for diverse data representations.

Support for comments
It allows adding comments for context or explanations within the file.

Purpose of Using YAML

Precise error handling
It helps the user in quickly finding and fixing errors.

Support for complex structures
It helps the user in referring to other data objects by supporting complex data structures.

Easy integration
It seamlessly integrates with version control systems and configuration management tools.

Cross-language support
It supports many programming languages, which makes it versatile for different development environments.

Where to Use YAML?

Configuring files

YAML is used to set up and adjust settings in applications due to its readability and user-friendly format.

Automation scripts

It is utilized in automation scripts and configuration management tools to define workflows and specify infrastructure settings.

Cloud infrastructure

It is employed to define Infrastructure as Code (IaC) in cloud environments, specifying resources and configurations effectively.

Data sharing

It facilitates sharing data between diverse programming languages and systems with varying data structures, ensuring seamless interoperability.

Comparison with Other Data Formats

Comparing features of YAML alongside trending data formats like XML and JSON:

Aspect	YAML	XML	JSON
Definition	A readable data serialization language	A markup language for structured data	A lightweight data interchange format
Readability	Highly readable with indentation	Requires explicit tags, less readable	Readable, but less than YAML
Human friendliness	Designed for human readability	Designed primarily for machine readability	Designed primarily for machine readability
Flexibility	Supports complex data structures	Limited flexibility, rigid hierarchy	Supports basic data structures
Usage	Widely used in configuration files and Ansible playbooks	Commonly used in markup languages and web services	Commonly used for data interchange

YAML, XML, and JSON Syntax: Example

Example: Consider a list of users with their name and age written in different formats.

Explanation: This example demonstrate how the same data is structured in each format, illustrating the readability and conciseness of YAML compared to XML and JSON.

YAML

```yaml
users:
  - name: John
    age: 30
  - name: Jane
    age: 25
```

XML

```xml
<users>
  <user>
    <name>John</name>
    <age>30</age>
  </user>
  <user>
    <name>Jane</name>
    <age>25</age>
  </user>
</users>
```

JSON

```json
{
  "users": [
    {
      "name": "John",
      "age": 30
    },
    {
      "name": "Jane",
      "age": 25
    }
  ]
}
```

Quick Check

You are tasked with choosing a data serialization format for configuring infrastructure as code (IaC). Considering the need for human readability, ease of writing, and support for complex data structures, which format should you choose, and why?

A. XML - widely used but verbose and harder to read.

B. JSON - lightweight and easy to parse but difficult to read when deeply nested.

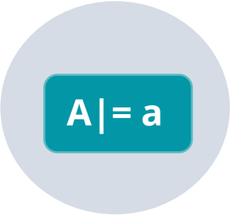
C. YAML - human-readable, easy to write, and supports complex data structures.

D. Terraform (.tf) - specific to Terraform, concise but not a general data format.

Exploring YAML Syntax

Basic Rules for YAML Syntax

It follows strict rules to avoid confusion and ensure consistent, reliable usage across different languages and editors.



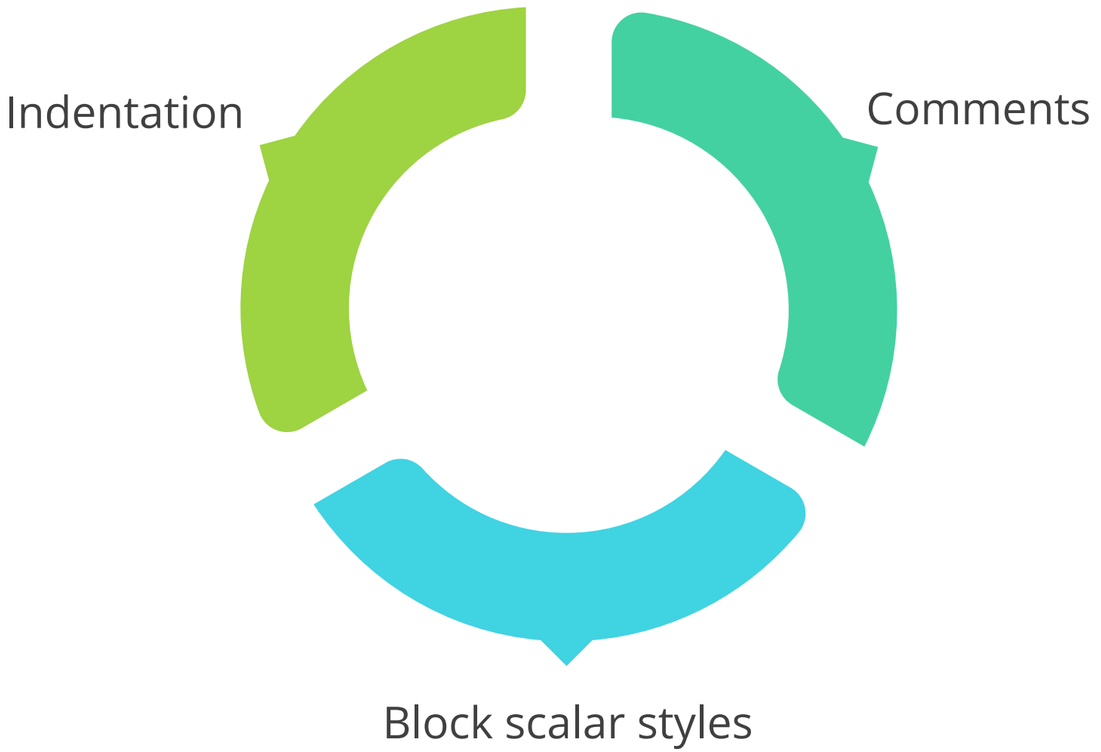
YAML is case-sensitive.

YAML files are saved with **.yml** or **.yaml** extension.

YAML does not allow the use of tabs while creating YAML files; however, spaces are allowed.

Key Concepts in YAML Syntax

To create and manage YAML files effectively, understanding the basics of YAML syntax is essential. Following are three fundamental aspects of YAML syntax:



YAML Syntax: Indentation

Indentation refers to the spaces or tabs added at the beginning of a line of text to visually indicate the level of nesting or hierarchy within a document or code.

Whitespace and indentation

YAML relies on whitespace and indentation to indicate nesting. The hierarchy and nesting are visible through a Python-like indentation style.

No tabs

Tab characters cannot be used for indentation in YAML files; only spaces can be used.

Consistency

The number of spaces used for indentation does not matter if they are consistent throughout the file.

YAML Syntax: Indentation

The following example shows the importance of consistent indentation in YAML:

```
tutorial:  # nesting level 1
  - yaml:  # nesting level 2 (2 spaces used for indentation)
      name: "YAML" # nesting level 3 (4 spaces used for indentation)
      type: awesome
      born: 2001
```

YAML Syntax: Comments

They are annotations or notes added within the YAML file to provide additional context, explanations, or remarks about the data or configuration being defined.

Comments are ignored by YAML parsers and have no impact on the data itself. They are preceded by the **#** character and extended until the end of the line.

YAML Syntax: Comments

The following code explains the use of comments in YAML files with the example of showcasing the company name:

```
# Comments inside a YAML file can be added followed by the # character
company: spacelift
```

YAML Syntax: Block Scalar Styles

For multiline strings, YAML provides block scalar styles. There are two types of block scalars:

1. Block literal style
2. Block folded style

Block Literal Style

It preserves newlines and leading whitespaces, allowing for a clean representation of multiline strings. In this example, the **|** denotes block literal style, and the content following **|** is treated as a multiline string.

Example:

```
data: |

 Every one

 Of these

 Newlines

 Will be

 Broken up.
```

Output:

```
Every one
Of these
Newlines
Will be
Broken up.
```

Block Folded Style

It is similar to block literal style, but it folds newlines into spaces unless a newline is followed by an empty or indented line. In this example, the **>** denotes the block folded style. The content following **>** is treated as a multiline string.

Example:

```
data: >
 This text
 is wrapped
 and will
 be formed into
 a single paragraph.
```

Output:

```
This text is wrapped and will be formed into a single paragraph.
```

What Does a Basic YAML Script Look Like?

Here is an example of a simple YAML script for employee records that follows the syntax rules:

```
---
#Employee Information

Employee-Id: 105672

Full-Name: Alice Smith

Department: Human Resources

Address:
 - street: 25 Oak Avenue
   city: New York
   state: NY

Position: Senior HR Specialist

Experience: |
  8 years in HR roles
  Certified in HR Management
...
```

Assisted Practice

Configuring Apache web server using Ansible **Duration: 10 Min.**

Problem Statement:

You've been assigned a task to configure and validate the setup of the Apache web server using Ansible on a local node machine for automated server management.

Assisted Practice: Guidelines

Steps to be followed:

1. Establish connectivity between the Ansible controller and node machine
2. Configure and validate the Apache web server setup

ASSISTED PRACTICE

Quick Check

You are configuring an Apache web server using Ansible and need to create a YAML playbook. Which YAML syntax practices will ensure that your playbook is correctly formatted and functions as intended?

A. Use tabs for indentation and place comments at the end of lines without spaces

B. Use spaces for indentation, ensure comments are preceded by a space, and use block folded style for multi-line strings

C. Maintain indent using either tabs or spaces, avoid comments, and use block literal style for single-line strings

D. Maintain indent using spaces, avoid comments, and use block folded style for single-line strings.

Key Takeaways

- YAML is an easy-to-read data serialization language often used to create configuration files with any programming language.
- YAML uses indentation to define the file structure, which makes the code easier to write and understand.
- YAML is case-sensitive, meaning it distinguishes between uppercase and lowercase letters.
- YAML seamlessly integrates with version control systems and configuration management tools, streamlining development workflows and enhancing productivity.
- YAML facilitates sharing data between diverse programming languages and systems with varying data structures. This ensures seamless interoperability and data exchange.

Thank You